

VOLUME 2

Architecture You Can Feel

*Responsibilities, feature boundaries, folder structure, dependency
direction, and growth*

A beginner-to-engineer rebuild guide grounded in the real repository

PATTERN > PRACTICE > REPETITION > REFINEMENT > HABIT

Learning Map

Use this volume as a build session, not passive reading. Type the examples, pause at checkpoints, and keep the architecture questions beside your editor.

Chapter	You will learn
1. Read the product as domains	Find stable areas of responsibility before choosing folders.
2. Feature-based structure	Place code by ownership and explain the compatibility layer.
3. Dependency direction	Keep shells, features, data, and shared code loosely coupled.
4. State ownership	Choose local state, hooks, or stores deliberately.
5. Refactoring without panic	Move boundaries gradually while the app keeps working.

Chapter 1: Read the Product as Domains

Chapter outcome: You will identify architecture from product behavior instead of starting with generic component folders.

WHAT A domain is a coherent area of product responsibility and language.

WHY Domains create boundaries that change for one reason and can be understood independently.

WHEN Look for them when features have their own state, vocabulary, workflows, or growth path.

WHERE In Macfolio they live mainly under src/features.

HOW Name the user-visible capability, its data, its decisions, and the other domains it must talk to.

Domain	Owns	Does not own
App composition	Startup, shell choice, top-level error/loading boundaries.	Feature internals.
Desktop shell	Dock, navbar, window configuration, window mechanics.	Finder file decisions.
Finder	Navigation UI and file-open decisions.	Raw GitHub requests.
Portfolio	Load and map portfolio project data.	Window rendering.
Code preview	Explorer, editor, terminal-like source experience.	Desktop window state.
Mobile shell	Phone navigation stack, app frame, mobile mini-apps.	Desktop window behavior.

Macfolio dependency story

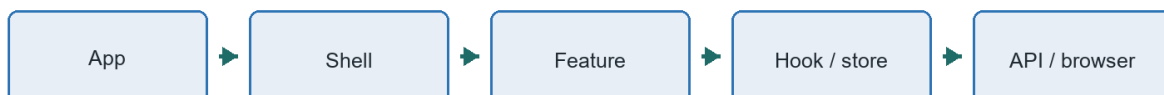


Figure 1. Dependencies should point toward narrower responsibilities, not loop between unrelated features.

Chapter 2: Feature-Based Structure

Chapter outcome: You will reproduce the repository structure and know what belongs in each folder.

Current high-level shape

```
src/
  app/                # application composition
  features/
    desktop-shell/    # desktop frame and window mechanics
    finder/           # Finder UI and open-item decisions
    portfolio/         # portfolio loading and mapping
    code-preview/      # explorer/editor/terminal experience
    mobile-shell/      # mobile navigation and mini-apps
  api/                # external service boundary
  constants/          # stable content grouped by ownership
  store/              # compatibility exports and cross-feature stores
  windows/            # desktop app windows and migration adapters
  components/         # app-level UI and compatibility exports
```

Why compatibility exports exist: Files such as `src/components/Dock.jsx` re-export the new feature implementation. This keeps old import paths working while architecture is moved gradually. They are migration bridges, not duplicate implementations.

- Belongs in a feature: components, hooks, utilities, data, styles, and tests used mainly by that capability.
- Belongs in app: providers, top-level shell choice, routing, and application bootstrapping.
- Belongs in api: raw remote requests and transport-level behavior.
- Belongs in shared only after unrelated features repeat the same stable contract.
- Does not belong in constants: helpers with side effects, React state, or API calls.

Chapter 3: Dependency Direction

Chapter outcome: You will recognize healthy imports, risky cycles, and boundaries that can scale to a team.

A simple dependency rule

```
App -> feature coordinator -> feature component / hook -> utility / API
```

```
Healthy: FinderWindow -> getFinderOpenAction
```

```
Risky:  github.js -> FinderWindow
```

```
Risky:  mobile app -> desktop window store
```

FinderWindow asks a pure utility what an item means, then performs the returned action. The utility never imports React, Zustand, or window.open. This makes the decision testable and prevents UI details from becoming the only place where product behavior is defined.

Actual pure action decision

```
import { isCodeFile, isExternalLinkFile } from "../fileTypes";
```

```

export const FINDER_OPEN_ACTIONS = {
  NONE: "none",
  SET_LOCATION: "set-location",
  OPEN_WINDOW: "open-window",
  OPEN_EXTERNAL_LINK: "open-external-link",
};

export const getFinderOpenAction = (item) => {
  if (!item) {
    return { type: FINDER_OPEN_ACTIONS.NONE };
  }

  if (item.fileType === "pdf") {
    return {
      type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
      windowId: "resume",
    };
  }

  if (item.kind === "folder") {
    return {
      type: FINDER_OPEN_ACTIONS.SET_LOCATION,
      location: item,
    };
  }

  if (item.fileType === "txt" && !item.download_url) {
    return {
      type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
      windowId: "txtfile",
      data: item,
    };
  }

  if (item.fileType === "img") {
    return {
      type: FINDER_OPEN_ACTIONS.OPEN_WINDOW,
      windowId: "imgfile",
      data: item,
    };
  }

  if (isExternalLinkFile(item.fileType) && item.href) {
    return {
      type: FINDER_OPEN_ACTIONS.OPEN_EXTERNAL_LINK,
      href: item.href,
    };
  }

  if (isCodeFile(item.fileType)) {

```

Three Ways To Understand It

Beginner explanation: A helper decides; the component performs the decision.

Developer explanation: The utility returns a small command object instead of triggering side effects.

Engineering explanation: Separating policy from mechanism lowers coupling and creates a stable unit for tests and future file types.

Chapter 4: State Ownership

Chapter outcome: You will decide whether state belongs in a component, a custom hook, a domain store, or persistence.

State type	Best owner	Macfolio example
One component only	useState in that component	Mobile navigation stack.
Reusable lifecycle logic	Custom hook	useWeather and useTodos.
Distant components in one domain	Feature store	Desktop windows.
Cross-feature navigation model	Focused store	Finder active location/history.
Persisted browser preference/data	Hook plus localStorage	Mobile todos.
Remote server data	Data hook/store with status	Portfolio Work folder.

Decision question: Which part of the product is allowed to change this value? Ownership is more useful than asking whether state is globally accessible.

Chapter 5: Refactoring Without Panic

Chapter outcome: You will move code toward feature boundaries in safe, explainable steps.

1. Document the current responsibility and imports.
2. Create the destination feature boundary.
3. Move the implementation without redesigning behavior.
4. Leave a small compatibility export at the old path.
5. Update direct imports gradually.
6. Verify lint, build, and user flow.
7. Remove the bridge only when no callers remain.

When not to refactor: Do not reorganize a stable area merely because a newer pattern looks fashionable. Refactor when ownership is unclear, change cost is rising, or a feature needs room to grow.

Checkpoint

- Beginner: Place a new desktop Calculator app in the structure.
- Intermediate: Choose ownership for shared project-card data used by desktop and mobile.
- Advanced: Design a migration from compatibility exports to direct feature imports without a flag day.